SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S.Mahammed Nayeem	**Reg. No.:**	192425294
Section / Batch:	AI&ML/2024	**Date:**	24-08-2026

Instructions to Students

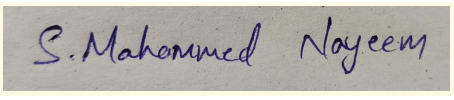
- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I S.Mahammed Nayeem/192425294(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Aim

To implement the Dice Throw Problem and Coin Change Problem using Dynamic Programming and compare their state transitions, recurrence relations, subproblem overlap, constraints, and computational complexity.

Procedure

1. Define the number of dice, number of faces, and target sum for the Dice Throw problem.
2. Create a DP table where each state represents the number of ways to obtain a particular sum using a given number of dice.
3. Initialize the base case with one way to obtain sum `0` using zero dice.
4. Calculate the number of dice-throw combinations using the recurrence relation.
5. Define a set of coins and the target amount for the Coin Change problem.
6. Create a DP array to calculate the number of ways to form each amount.
7. Use previously calculated subproblems to obtain the final result.
8. Implement another DP approach to find the minimum number of coins required.
9. Compare both problems based on state transitions, constraints, recurrence relations, and complexity.
10. Display the results for the given input.

Python Code

```
def dice_throw_ways(dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(dice + 1)]
    dp[0][0] = 1

    for i in range(1, dice + 1):
        for j in range(1, target + 1):
            for f in range(1, faces + 1):
                if j - f >= 0:
                    dp[i][j] += dp[i - 1][j - f]

    return dp[dice][target]


def coin_change_ways(coins, amount):
    dp = [0] * (amount + 1)
    dp[0] = 1

    for coin in coins:
        for j in range(coin, amount + 1):
            dp[j] += dp[j - coin]

    return dp[amount]
```

```
def coin_change_min_coins(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for j in range(1, amount + 1):
        for coin in coins:
            if j - coin >= 0 and dp[j - coin] != float('inf'):
                dp[j] = min(dp[j], dp[j - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1


# Dice Throw
dice = 2
faces = 6
target = 7

dice_result = dice_throw_ways(dice, faces, target)

# Coin Change
coins = [1, 2, 5]
amount = 7

ways = coin_change_ways(coins, amount)
minimum = coin_change_min_coins(coins, amount)

print("Dice Throw: Ways to get", target)
print("using", dice, "dice with", faces, "faces each =", dice_result)

print("\nCoin Change: Ways to form amount", amount)
print("using", coins, "=", ways)

print("Coin Change: Minimum coins to form", amount)
print("using", coins, "=", minimum)
```

Input

Dice Throw:
dice = 2
faces = 6
target = 7

Coin Change:
coins = [1, 2, 5]

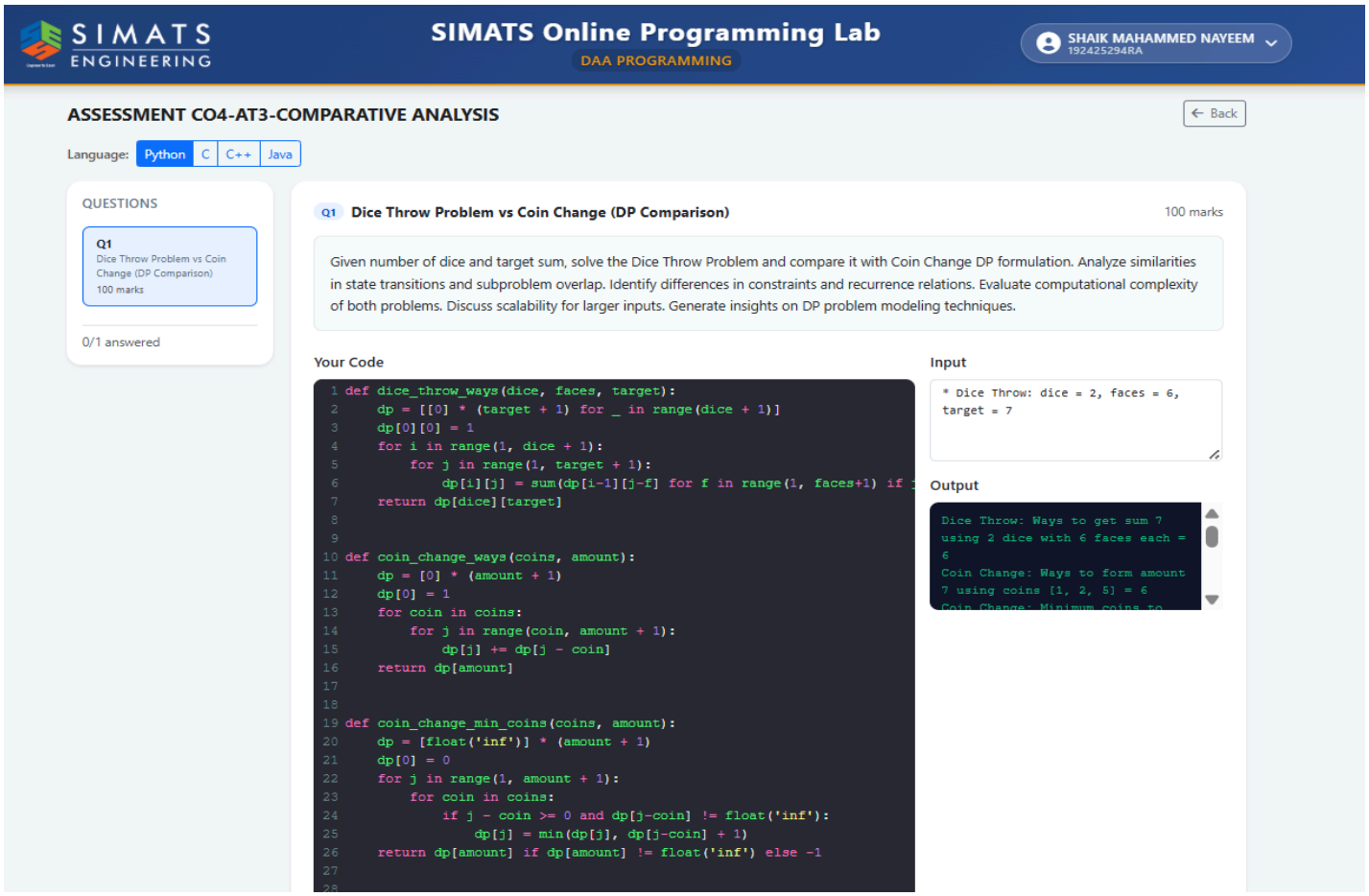
amount = 7

Output

Dice Throw: Ways to get 7
using 2 dice with 6 faces each = 6

Coin Change: Ways to form amount 7
using [1, 2, 5] = 6

Coin Change: Minimum coins to form 7
using [1, 2, 5] = 2



Result

The Dice Throw and Coin Change problems were successfully implemented using Dynamic Programming. Both problems demonstrate optimal substructure and overlapping subproblems, but their state transitions and constraints differ. For the given input, the Dice Throw problem has 6 ways to obtain a sum of 7, while Coin Change has 6 ways to form amount 7 and requires a minimum of 2 coins.